COMP 4958: Assignment 1

Submission deadline will be announced. You will need to submit a zip file containing the project folder named `chat` (excluding the `_build` directory). Be sure to comment your source files. You will need to set up and demonstrate your "chat" system, at which time, it will be tested and you will need to explain and answer questions about your code.

For this assignment, you are asked to implement a system in Elixir that allows users to send messages to one another.

The user connects to the system via TCP and issues commands. The following table shows the valid commands with brief descriptions:

Command	*Brief description*
/B <message>	Broadcast message to default group
/D <names>	Set default group to users with specified names
/G <group_name> <names>	Assign group name to users specified by names
/L	List current users
/M <names> <message>	Send message to recipients specified by names
/N <nick_name>	Set a nickname

Each user input line must essentially be one of the commands listed above. However, commands are case-insensitive and we also accept variants: as long as the first word in the input line starts with the 2-letter commands listed above (in either uppercase or lowercase), it will be accepted. For example, /m and /msg are regarded as variants of the /M command. Each command is terminated by the end-of-line character sequence. From the table, we see that different commands may take different number of arguments. Extra arguments/words are always ignored. For example, `/L hell` is valid; `hell` is simply ignored! Note that any input line that is not one of the 6 commands listed above is invalid.

There are two types of names that we need to deal with: nick names and group names. A (valid) nickname must start with a *lowercase* alphabet, followed by characters that are either alphanumeric or the underscore character, and its length must be between 2 and 20 characters inclusive. A group name has the same requirements as a nick name, except that it must start with an *uppercase* alphabet. In the above table, <names> stands for a "list" of nick names and/or group names. More details below.

The /N command is used to set a nick name. It takes one word as argument; any extra words are ignored (as mentioned above). Hence a /N command with no following words is invalid. For example, `/N homer` asks the server to use `homer` as the nick name for the user who issues that command. The server needs to ensure that the requested nick name is not in use before granting the request. The client receives a response message indicating whether the operation is successful. Note the requirements for nick names mentioned above. This means the word that follows /N (or its variants) must be validated. (This should be done before the request is sent to the server.) A user must succeed in setting a nick name before they are allowed to issue any other command except the /L command: the user can always use the /L command to get a list of the nick names currently in use. A user can also use the /N command to change their existing nick name. In the following, if necessary, we will refer to a user that has acquired a nick name as a registered user.

The /M command is used to send a message to a specific user or a list of users (which may be specified via group names; see /G command below.) `/M homer hello world` sends the message "hello world" to the user with nick name `homer`. `/M homer,bart hello world` sends the message to users `homer` and `bart`. Note that the nick names are separated by commas. Clearly, the /M command must be followed by at least two words: the first word specifies the recipient(s) and the rest the actual message. A /M command followed by fewer than two words is invalid. Note that only registered users can send (and receive) messages.

The /G command is used to assign a group name to a list of (one or more) names. After `/G Simpsons homer,bart,lisa`, we can use, e.g., `/M Simpsons,monty hello there` to send the message "hello there" to the 4 users: homer, bart, lisa and monty. Clearly, the /G command must be followed by at least two words: the first word specifies the group name and the second word the members of the group. Note that the /G command can use other group names, e.g., `/G SF marge,Simpsons`. In this example, if the `Simpsons` group name had not existed before, it is regarded as an empty list and `SF` will be just `marge`. As another example, `/G Simpsons Simpsons,marge` adds `marge` to the `Simpsons` group. Note that group names are specific to a chat client: group names set by one chat client are not available to another chat client.

The /D command is used to designate a list of names as the default group. The default group is used by the /B command, which sends a message to the default group. For example, after issuing `/D homer,bart`, `/B hello world` sends the message "hello world" to homer and bart.

For the implementation, there must be one globally-registered supervised server that provides the main behaviour of dispatching messages and handling nick names, i.e., it receives requests from other processes whenever messages need to be sent and whenever a user needs to set or change their nickname. This server must use one or more ETS tables to keep track of its state. We'll refer to this server as the chat server. Put its implementation in a a module named `Chat.Server`. This server does not directly communicate with external chat clients. External chat clients connects to the chat system using TCP and may be implemented in a language different from Elixir. An external client is represented by an Elixir worker process that we call a "proxy worker". It is this worker process that communicate with the server on behalf of the client.

The peoxy worker is responsible for parsing and validating commands sent by the connected client, remembering and handling group names and the default group for the client. This reduces the workload on the server. The proxy worker requests the chat server for services when needed, e.g., to send a message to users, or to register a nick name. It also forwards messages

from the chat server, as well as sends its own error messages (e.g., for invalid commands), to its connected client. From the point of view of the external TCP client, the worker process is a proxy for the chat server; from the point of view of the server, the worker process is a proxy for the external client. Put the code for this worker in a module named `Chat.Proxy.Worker`.

There must be a third type of processs: we need a process that creates the worker processes. We call such a process a "chat proxy". Put its implementation in a module named `Chat.Proxy`. Its only purpose is to create proxy workers. It is not registered and we can run different instances of it on different nodes.

Your mix project should be named `chat`. From the description above, it is clear that you need to implement at least 3 modules. All 3 modules use GenServer. You may introduce additional helper modules if necessary. Note that `Chat.Proxy` implements a process that listens at a particular port (defaults to 6666) and creates a proxy worker whenever an external client connects to the system via TCP.

You will also need to implement a suitable client either in Elixir or in Java. It takes a port number and a host as command-line arguments. (The port defaults to 6666 and the host defaults to "localhost".) Basically, the program needs to connect to a proxy, sends commands the user types to the proxy worker that is created, as well as receiving and displaying replies from that worker concurrently. It is a "dumb" client and does not do command validation or remember group names. The program terminates when the user enters the end-of-file key at the beginning of an input line. The chat system needs to remove the nickname when a client terminates.

The system needs to be fairly user-friendly and appropriate error messages should be returned to the client in certain cases. To faciliate testing, the system needs to print some "debugging" information, e.g, who has connected and disconnected, etc.